PROJETO TRANSPORTE SOCIAL

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP 1º BIMESTRE)

Kenzo Wilder Yamamoto - R.A: 25247917-2.

Sam Gomes Cangira Osako - R.A: 25000548-2

Instituição: Unicesumar (Maringá - PR)

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

Maria tem 52 anos, trabalha como diarista e mora em um bairro afastado do centro. A renda da família dela gira em torno de R$ 1.600,00 por mês. Para chegar ao trabalho ou ir até o posto de saúde, ela depende de ônibus que passam de hora em hora e fazem um trajeto longo. Quando o horário não bate, a saída é pagar uma corrida de aplicativo, que custa uns R$ 25,00. Esse valor pesa muito no orçamento da semana, então na maioria das vezes ela acaba desistindo de sair.

O sistema que propomos é um aplicativo de corridas parecido com o Uber, só que simplificado e voltado apenas para quem tem renda familiar abaixo de R$ 2.000,00. A prefeitura paga metade do valor da corrida, com teto de R$ 15,00, e cada cidadão tem direito a 5 viagens por semana. Antes de liberar o acesso, a prefeitura confere os documentos do cidadão e do motorista, inclusive a certidão de antecedentes criminais, para dar segurança aos dois lados.

O projeto está ligado à **ODS 10 da ONU (Redução das Desigualdades)**, porque dá acesso a um transporte rápido e seguro para quem antes não tinha como pagar, facilitando a ida ao trabalho, ao médico e à escola. Ele também toca a **ODS 11 (Cidades e Comunidades Sustentáveis)**, na meta 11.2, que fala em transporte seguro e a preço acessível para todos, principalmente para as pessoas em situação de vulnerabilidade.

Resumo do Discovery (Matriz CSD e Jornada)

Na fase de descoberta montamos a Matriz CSD. Nas **certezas**, ficou claro que o público-alvo tem pouca renda, depende de um transporte público com horários ruins e que a prefeitura precisa saber quem recebe o benefício e quanto está sendo gasto. Nas **suposições**, partimos do princípio de que 5 viagens por semana dão conta dos deslocamentos mais importantes, como a ida e a volta do trabalho ou uma consulta, e que os motoristas vão querer atender esse público porque parte do valor já vem garantida. A maior **dúvida** foi como comprovar renda, idade e antecedentes sem depender de integração com sistemas do governo, que seria complexa demais para o prazo do projeto. Optamos por pedir os documentos no cadastro e deixar a conferência com um administrador da prefeitura.

A jornada do cidadão ficou com cinco passos: cadastro com envio dos documentos, aprovação pela prefeitura, solicitação da viagem já vendo quanto vai pagar, acompanhamento do motorista até o destino e pagamento da sua parte por crédito, débito ou Pix.

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

- **RF01** – O sistema deve permitir cadastrar, consultar, alterar e excluir usuários, identificando se o usuário é cidadão ou motorista.
- **RF02** – O sistema deve permitir o envio dos documentos exigidos no cadastro: CPF e comprovante de renda para o cidadão, CNH para o motorista e certidão de antecedentes criminais para os dois (RN01 e RN03).
- **RF03** – O sistema deve permitir que o administrador da prefeitura consulte os cadastros pendentes, aprove ou recuse cada um e registre o motivo da recusa (RN04).
- **RF04** – O sistema deve bloquear as funções de viagem enquanto o cadastro do usuário não estiver aprovado (RN04).
- **RF05** – O sistema deve permitir que o motorista cadastre e altere os dados do seu veículo (placa, marca, modelo, ano e cor).
- **RF06** – O sistema deve permitir que o motorista informe se está disponível ou indisponível para receber viagens.
- **RF07** – O sistema deve permitir que o cidadão solicite uma viagem informando origem, destino e quantos menores vão acompanhá-lo (RN02).
- **RF08** – O sistema deve calcular e mostrar, antes da confirmação da viagem, o valor total, o valor pago pela prefeitura e o valor que o cidadão vai pagar (RN06 e RN07).
- **RF09** – O sistema deve permitir que o motorista disponível veja as solicitações em aberto e aceite uma delas.
- **RF10** – O sistema deve mostrar ao cidadão o status da viagem e o tempo estimado de chegada do motorista.
- **RF11** – O sistema deve registrar o pagamento da parte do cidadão por cartão de crédito, cartão de débito ou Pix, guardando também o valor do subsídio que a prefeitura vai repassar (RN08).
- **RF12** – O sistema deve informar quantas viagens ainda restam na semana e impedir novas solicitações quando o limite for atingido (RN05).
- **RF13** – O sistema deve permitir que o cidadão e o motorista consultem o histórico das suas viagens.
- **RF14** – O sistema deve permitir que o administrador gere um relatório com as viagens realizadas e o total de subsídio pago em um período.

Requisitos Não Funcionais (RNF)

- **RNF01** – **Segurança:** as senhas são gravadas com hash (BCrypt) e cada tela só é liberada para o perfil correspondente (cidadão, motorista ou administrador).
- **RNF02** – **Privacidade (LGPD):** os documentos enviados no cadastro só podem ser abertos pelo administrador da prefeitura e são usados apenas para a análise de elegibilidade.
- **RNF03** – **Usabilidade:** o sistema roda no navegador do celular, com layout responsivo, porque a maior parte do público acessa pelo telefone.
- **RNF04** – **Desempenho:** a tela de solicitação deve mostrar o valor calculado da viagem em até 3 segundos.

- **RNF05** – **Disponibilidade:** o sistema deve ficar no ar das 5h às 23h, que é a faixa de horário prevista para as viagens.
- **RNF06** – **Compatibilidade:** funcionar nas versões atuais do Chrome, do Firefox e do Edge.
- **RNF07** – **Manutenibilidade:** o back-end é dividido em camadas (modelo, persistência e controle), separando as regras de negócio do acesso ao banco.

Regras de Negócio (RN)

- **RN01** – **Elegibilidade do cidadão:** renda familiar bruta mensal abaixo de R$ 2.000,00, idade mínima de 18 anos e nenhum antecedente criminal.
- **RN02** – **Menores de idade:** menores de 18 anos não têm cadastro próprio e só viajam junto com um responsável cadastrado, que informa quantos menores vão na hora de pedir a viagem.
- **RN03** – **Motorista:** CNH válida com a observação EAR (Exerce Atividade Remunerada), ficha limpa e nenhum processo em aberto por crime ou por conduta que coloque o passageiro em risco.
- **RN04** – **Aprovação do cadastro:** todo cadastro nasce com o status PENDENTE e só passa para APROVADO depois da análise do administrador. O cadastro recusado recebe o motivo e pode ser refeito com novos documentos.
- **RN05** – **Limite semanal:** no máximo 5 viagens por cidadão por semana, contando de segunda a domingo. Só entram na conta as viagens concluídas, e os menores acompanhantes não contam como viagem separada.
- **RN06** – **Subsídio:** a prefeitura paga 50% do valor da viagem, com teto de R$ 15,00. Até R$ 30,00 ela paga a metade; acima disso paga R$ 15,00 e o restante fica com o cidadão.
- **RN07** – **Valor da viagem:** tarifa base mais um valor por quilômetro rodado, os dois definidos pela prefeitura (exemplo: R$ 5,00 + R$ 2,00 por km).
- **RN08** – **Pagamento:** só cartão de crédito, cartão de débito ou Pix, e apenas sobre a parte do cidadão. Não existe pagamento em dinheiro.
- **RN09** – **Cancelamento:** o cidadão pode cancelar sem custo enquanto o motorista não chegou ao local de embarque, e a solicitação que ninguém aceitar em 10 minutos é cancelada pelo sistema. Viagem cancelada não gera subsídio nem cobrança.

Exemplos de cálculo do subsídio

Fórmula: subsídio = o menor valor entre (valor da viagem x 50%) e R$ 15,00. Valor pago pelo cidadão = valor da viagem - subsídio.

Valor da viagem	Prefeitura paga (subsídio)	Cidadão paga
R$ 20,00	R$ 10,00	R$ 10,00
R$ 30,00	R$ 15,00	R$ 15,00
R$ 40,00	R$ 15,00 (teto)	R$ 25,00

3. O PLANEJAMENTO ÁGIL (CRONOGRAMA / BACKLOG)

Abaixo, apresenta-se o fatiamento vertical das entregas simulando o Roadmap de desenvolvimento do 2º Bimestre.

Sprint / Data	Épico	User Story	Responsável
Sprint 1 05/09 - 19/09	Cadastro de usuários (RF01, RF02 e RF05)	**COMO UM** cidadão de baixa renda **EU QUERO** me cadastrar enviando meus documentos (CPF, comprovante de renda e antecedentes criminais) **PARA QUE** eu possa ter acesso ao transporte subsidiado. **COMO UM** motorista **EU QUERO** me cadastrar com minha CNH, antecedentes criminais e os dados do meu veículo **PARA QUE** eu possa realizar viagens pelo sistema.	Kenzo Wilder Yamamoto
Sprint 2 20/09 - 04/10	Aprovação de cadastros (RF03 e RF04)	**COMO UM** administrador da prefeitura **EU QUERO** analisar os documentos e aprovar ou recusar cadastros de cidadãos e motoristas **PARA QUE** apenas pessoas elegíveis e com ficha limpa utilizem o sistema.	Sam Gomes Cangirana Osako
Sprint 3 05/10 - 19/10	Solicitação e aceite de viagem (RF06 a RF10)	**COMO UM** cidadão aprovado **EU QUERO** solicitar uma viagem vendo o valor total, o desconto da prefeitura e quanto vou pagar **PARA QUE** eu possa me locomover pela cidade pagando menos. **COMO UM** motorista **EU QUERO** visualizar e aceitar as solicitações de viagem **PARA QUE** eu possa atender os passageiros.	Kenzo Wilder Yamamoto
Sprint 4 20/10 - 03/11	Pagamento e limite semanal (RF11 a RF14)	**COMO UM** cidadão **EU QUERO** pagar a minha parte por crédito, débito ou Pix e ver quantas viagens ainda posso fazer na semana **PARA QUE** eu controle o uso do meu benefício. **COMO UM** administrador **EU QUERO** gerar um relatório de viagens e subsídios pagos **PARA QUE** a prefeitura acompanhe os gastos do programa.	Sam Gomes Cangirana Osako

4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

No front-end usamos **HTML** para a estrutura das telas, **CSS** para o visual e **JavaScript** para a parte interativa. O back-end é feito em **Java**, escolhido porque o curso trabalha com Programação Orientada a Objetos e a linguagem deixa o sistema bem dividido em classes (Usuario, Cidadao, Motorista, Viagem e Pagamento), o que facilita os testes e a manutenção. A troca de informações entre as duas partes é feita por requisições HTTP com JSON.

O back-end fica separado em camadas: as classes de modelo, com os dados e as regras; uma camada de persistência (DAO), que conversa com o banco; e uma camada de controle, que recebe as requisições. Os valores em dinheiro usam **BigDecimal** no lugar de double, porque o double gera erro de arredondamento em conta de centavo, e as senhas são gravadas com hash BCrypt, nunca em texto puro.

Os dados ficam em um banco relacional **MySQL**, administrado pelo **phpMyAdmin**, que é

onde criamos as tabelas e os relacionamentos. As regras que não podem ser burladas, como o cálculo do subsídio e o limite de 5 viagens na semana, são conferidas no back-end antes de gravar, e não apenas na tela.

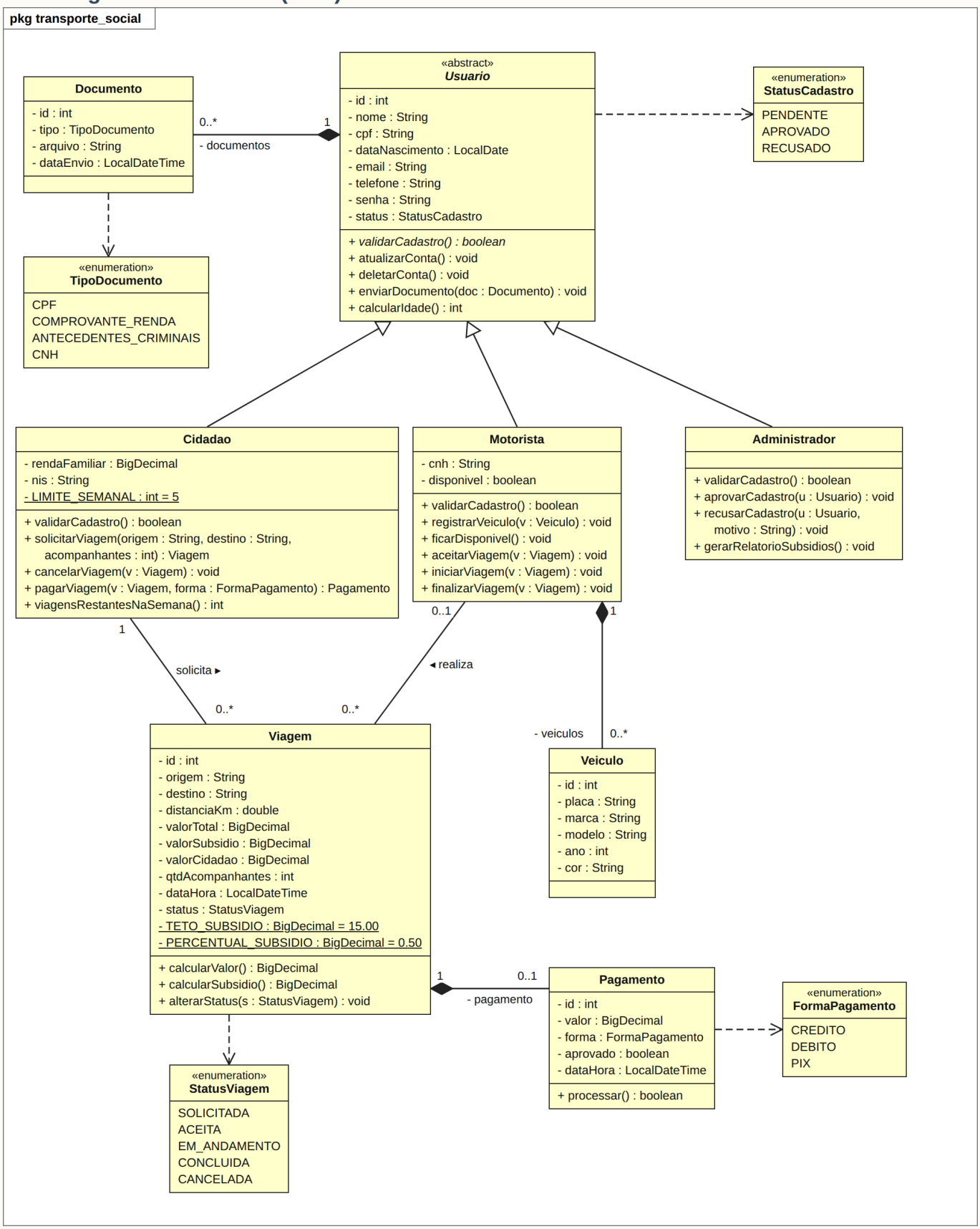
Para o projeto não ficar grande demais, deixamos algumas coisas **fora do escopo**: viagem compartilhada, tarifa dinâmica, avaliação de motorista e pagamento em dinheiro. A distância pode vir de uma API de mapas (Google Maps ou OpenStreetMap) e, na primeira versão, ela é estimada a partir dos endereços digitados. O pagamento por cartão e por Pix também é simulado, sem integração com instituição financeira.

5. ESTRUTURAÇÃO DO GITHUB E DIAGRAMAS

5.1 Link do Repositório

- **Repositório Público GitHub:** https://github.com/kenzowy/AEP-4-Semestre
- **Estrutura do repositório:** O repositório é público e já está com as pastas criadas para o 2º bimestre: /src para o código-fonte, /docs para a documentação e os diagramas e /database para o script .sql do banco, além do README.md com as instruções de execução.

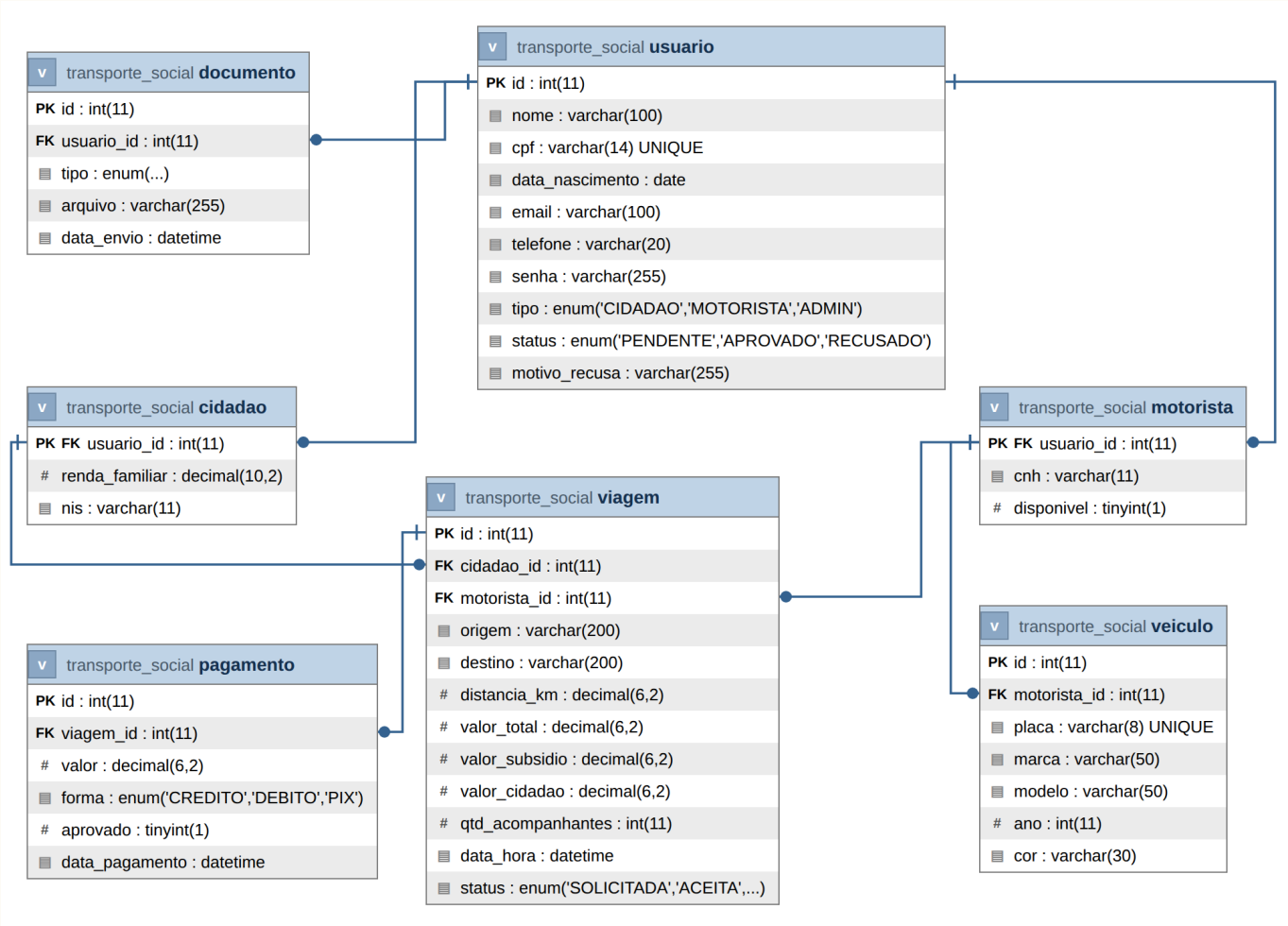
5.2 Diagrama de Classes (UML)



- **Herança:** A superclasse abstrata Usuario guarda o que é comum a todos (id, nome, cpf, dataNascimento, email, telefone, senha e status do cadastro). Dela herdam Cidadao, Motorista e Administrador, cada uma com os seus próprios métodos.
- **Composição 1:N:** O Motorista é dono dos seus Veiculos e o Usuario é dono dos seus Documentos. Se o usuário for excluído, os veículos e os documentos dele somem junto. O Pagamento segue a mesma ideia: só existe ligado a uma Viagem.

- **Associação:** Um Cidadao solicita várias Viagens (1:N) e cada Viagem é realizada por no máximo um Motorista (0..1), porque a viagem já existe enquanto ainda não foi aceita por ninguém.
- **Polimorfismo:** O método abstrato validarCadastro() é sobrescrito (@Override) de um jeito diferente em cada subclasse: em Cidadao ele confere a idade mínima de 18 anos e a renda abaixo de R$ 2.000,00; em Motorista confere a CNH com EAR e a certidão de antecedentes; em Administrador só permite a criação pela própria prefeitura.
- **Encapsulamento:** Todos os atributos são privados e o acesso é feito por getters e setters. As regras fixas viram constantes dentro da classe que as usa, como LIMITE_SEMANAL em Cidadao e TETO_SUBSIDIO e PERCENTUAL_SUBSIDIO em Viagem, que são do tipo BigDecimal por serem valores em dinheiro.
- **Enumerações e dependência:** StatusCadastro, TipoDocumento, StatusViagem e FormaPagamento se ligam por dependência (seta tracejada) às classes que as usam como tipo de atributo: Usuario, Documento, Viagem e Pagamento. Elas evitam que um valor inválido entre no sistema, como uma forma de pagamento diferente de CREDITO, DEBITO e PIX.

5.3 Diagrama de Banco de Dados (DER)



- **Tabela usuario:** Guarda os dados comuns a todos os usuários, o tipo (cidadão, motorista ou administrador) e o status do cadastro. O cpf é único e a coluna senha guarda o hash, não a senha digitada.
- **Tabelas cidadao e motorista:** Guardam os dados específicos de cada tipo de usuário. As duas usam o usuario_id como chave primária e estrangeira ao mesmo tempo, que é a

forma de representar a herança no banco.

- **Tabela documento:** Guarda os documentos enviados no cadastro, ligados ao usuário pela chave estrangeira usuario_id.
- **Tabela veiculo:** Guarda os veículos de cada motorista, com a chave estrangeira motorista_id e a placa como campo único.
- **Tabela viagem:** Guarda origem, destino, valores (total, subsídio e parte do cidadão), acompanhantes e status, ligando o cidadão e o motorista. Para contar o limite da semana criamos um índice em (cidadao_id, status, data_hora), que é a consulta mais usada do sistema.
- **Tabela pagamento:** Guarda o pagamento da parte do cidadão e a forma escolhida (crédito, débito ou Pix). Os valores em dinheiro usam decimal, e não float, para não perder centavos no arredondamento.